T.C.

FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ

Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

Lisans Bitirme Projesi

LLM AJANLARI İLE YAPAY ZEKÂ TABANLI ETKİLEŞİMLİ YAZILIM GELİŞTİRME PLATFORMU

Ömer Yasir ÖNAL – 2221221562

Ayşe Serra GÜMÜŞTAKIM - 2121251008

Bitirme Projesi Danışmanı: Dr. Öğr. Üyesi Nazlı DOĞAN

İstanbul, Ocak 2026

T.C.

FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ

Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

Lisans Bitirme Projesi

LLM AJANLARI İLE YAPAY ZEKÂ TABANLI ETKİLEŞİMLİ YAZILIM GELİŞTİRME PLATFORMU

Ömer Yasir ÖNAL – 2221221562

Ayşe Serra GÜMÜŞTAKIM - 2121251008

Bitirme Projesi Danışmanı: Dr. Öğr. Üyesi Nazlı DOĞAN

Jüri Üyeleri:	İmza:
……..:	.....
……..:	.....
……..:	.....

İstanbul, Ocak 2026

İÇİNDEKİLER

ÖN SÖZ **5**

ÖZET **6**

SUMMARY **7**

ŞEKİL LİSTESİ **8**

TABLO LİSTESİ **9**

1. GİRİŞ **10**

1.1. PROBLEM TANIMI VE MOTİVASYON 10
1.2. PROJENİN KAPSAMI 10
1.3. TERİMLER VE KISALTMALAR 11

2. İLGİLİ ÇALIŞMALAR / LİTERATÜR ARAŞTIRMASI **12**

2.1. ARAŞTIRMA METODOLOJİSİ 12
2.2. BULGULARIN ÖZETİ VE AKIS İLE İLİŞKİSİ 12

3. PROJE GEREKSİNİMLERİ **13**

3.1. FONKSİYONEL GEREKSİNİMLER 13
3.2. FONKSİYONEL OLMAYAN GEREKSİNİMLER 14
3.3. TASARIM KISITLARI VE VARSAYIMLAR 14

4. SİSTEM TASARIMI **15**

4.1. GENEL MİMARİ BAKIŞ 15
4.2. BACKEND TASARIMI 16
4.2.1. AgentOrchestrator ve Ajan Yaşam Döngüsü *16*
4.2.2. Kimlik Doğrulama ve Onboarding Akışları *17*
4.2.3. API Katmanı ve Uç Nokta Özeti *17*
4.2.4. Güvenlik Katmanı ve Orta Katmanlar *18*
4.2.5. Modül Yapısı ve Dosya Organizasyonu *18*
4.3. FRONTEND TASARIMI 18
4.3.1. Bilgi Mimarisi ve Ekranlar *18*
4.4. VERİ MODELİ TASARIMI 18
4.4.1. Varlıklar ve İlişkisel Model Özeti *19*
4.4.2. Job, İz Kaydı ve Artefakt Yaşam Döngüsü *19*
4.4.3. Tablo Alanları (Kolon Bazında) *20*
4.5. ENTEGRASYON TASARIMI 29
4.5.1. MCP Gateway ve Adaptör Yaklaşımı *29*
4.5.2. Docker Bileşenleri ve Çalıştırma Modeli *30*
4.5.3. Entegrasyon Güvenliği ve Erişim Kontrolleri *30*

4.6. TEST PLANI VE KALİTE STRATEJİSİ 30
4.7. DAĞITIM VE İŞLETİM 31

5. TAMAMLANAN GÖREVLER 31

5.1. YAPILAN İŞLER VE ARA ÇIKTILAR 31
5.2. GÜNCEL DURUM TABLOSU 32

KAYNAKLAR 32

ÖN SÖZ

Bu rapor, Fatih Sultan Mehmet Vakıf Üniversitesi Bilgisayar Mühendisliği Bölümü Lisans Bitirme Projesi kapsamında geliştirilen **AKIS Platformu**'nun çözüm tasarımı ve teknik analizini ayrıntılı biçimde sunmaktadır. Çalışmanın temel amacı; yazılım geliştirme yaşam döngüsünde (SDLC) dokümantasyon üretimi, iz kayıtları üzerinden inceleme, iş akışı görünürlük/izlenebilirlik ihtiyacı ve dış sistem entegrasyonları gibi operasyonel yükü yüksek süreçlerde, **LLM tabanlı ajan yaklaşımı** ile verimlilik ve denetlenebilirlik sağlayan bir prototip ortaya koymaktır. Bu kapsamda AKIS; merkezi bir orkestrasyon katmanı altında çalışan ajanlar, güvenli entegrasyon adaptörleri ve kalıcı iz/artefakt yönetimi ile uçtan uca bir iş yürütme çerçevesi sunmayı hedefler.

Bu süreçte sağladığı değerli yönlendirmeler ve katkıları için proje danışmanımız **Dr. Öğr. Üyesi Nazlı DOĞAN**'a teşekkür ederiz. Ayrıca proje geliştirme sürecindeki iş birliği ve emekleri için ekip üyelerine teşekkür ederiz.

Saygılarımızla,
Ömer Yasir Önal
04.01.2026

AKIS PLATFORMU: LLM TABANLI AJANLAR İLE ETKİLEŞİMLİ YAZILIM GELİŞTİRME VE YÖNETİŞİM SİSTEMİ

ÖZET

Bu doküman, AKIS Platformu'nun çözüm tasarımını ve teknik analizini sunmaktadır. AKIS; yazılım geliştirme yaşam döngüsünde (SDLC) tekrarlayan ve düşük katma değerli işleri azaltmayı hedefleyen, LLM tabanlı ajanlar üzerinden etkileşimli çalışan bir yazılım platformu prototipidir. Platform; modüler monolit bir backend (Fastify + TypeScript) ve bağımsız bir SPA frontend (React + Vite) yaklaşımıyla tasarlanmış, job yürütme sonuçlarını, iz kayıtlarını ve üretilen artefaktları PostgreSQL üzerinde Drizzle ile kalıcılaştırmıştır.

Mevcut aşamada; AgentOrchestrator çekirdeği, job yaşam döngüsü durum makinesi, çok adımlı kayıt/giriş ve e-posta doğrulama akışları ile dashboard üzerinden ajan konfigürasyonu ve demo senaryolarını çalıştırma deneyimi oluşturulmuştur. Tasarım; doğrulanabilirlik (iz ve artefakt kaydı), güvenlik kontrolleri (oran sınırlama, güvenli oturum çerezi), düşük bağımlılık ve sınırlı kaynaklı bulut ortamlarında çalışabilirlik hedefleri etrafında şekillendirilmiştir. Doküman ayrıca literatürdeki LLM ajan yaklaşımlarını özetler ve AKIS tasarım kararlarını proje gereksinimleriyle ilişkilendirir.

SUMMARY

This document presents the solution design and technical analysis of the AKIS Platform, an interactive LLM-agent–based software delivery assistant. The platform aims to reduce repetitive SDLC activities such as documentation generation, trace/log inspection, and standardized integrations with external developer tools. AKIS is designed as a modular monolith backend (Fastify + TypeScript) with a decoupled SPA frontend (React + Vite), and persists jobs, traces, and artifacts in PostgreSQL via Drizzle.

At the current stage, the core orchestration layer (AgentOrchestrator) and the job lifecycle state machine have been established, together with multi-step authentication flows and an initial dashboard experience to configure and run agent jobs for demo scenarios. The architecture prioritizes auditability, security controls (rate limiting, secure session cookies), and resource efficiency to run on constrained cloud environments. The document also summarizes related work on autonomous agents and reasoning-acting patterns, and explains key design decisions with respect to project requirements.

ŞEKİL LİSTESİ

Şekil 4.1: AKIS Platformu genel mimari bileşenleri (backend–frontend–veritabanı–MCP entegrasyonları).
Şekil 4.2: Job yaşam döngüsü için durum makinesi (FSM) özeti.

TABLO LİSTESİ

Tablo 3.1: Fonksiyonel Gereksinimler (FR) özeti.
Tablo 3.2: Fonksiyonel Olmayan Gereksinimler (NFR) özeti.
Tablo 4.1: Veritabanı tabloları ve amaçları (özet).
Tablo 4.2: jobs tablosu alanları (seçilmiş kolonlar).
Tablo 4.3: job_traces tablosu alanları (seçilmiş kolonlar).
Tablo 5.1: Güncel durum tablosu (fazlar ve odak alanları).

1. GİRİŞ

1.1. Problem Tanımı ve Motivasyon

Günümüz yazılım geliştirme süreçleri (Software Development Life Cycle - SDLC), artan kod tabanı karmaşıklığı, sık değişen gereksinimler ve hızlanan teslim beklentileri nedeniyle ekipler için ciddi operasyonel yükler doğurmaktadır. Bu projede, yazılım ekiplerinin verimliliğini düşüren ve maliyetlerini artıran üç problem alanı hedeflenmiştir: (i) eskimiş dokümantasyon, (ii) yüksek test otomasyon maliyeti ve (iii) yavaş MVP prototipleme.

AKIS Platformu; tekrarlayan ve düşük katma değerli SDLC görevlerini LLM tabanlı ajanlar üzerinden otomatikleştirerek, ekiplerin yüksek katma değerli mühendislik kararlarına ve ürün geliştirme faaliyetlerine odaklanmasını amaçlar. Platform; dokümantasyon güncelleme (AKIS Scribe), test otomasyonu üretimi (AKIS Trace) ve gereksinimlerden çalışan prototip çıkarma (AKIS Proto) fonksiyonlarını tek bir orkestrasyon çekirdeği altında birleştirir.

1.2. Projenin Kapsamı

AKIS Platformu, tek repo içinde modüler monolit bir backend ve bağımsız bir SPA frontend yaklaşımıyla tasarlanmıştır. Sistem; ajan yürütme yaşam döngüsünü yöneten merkezi bir düzenleyici (AgentOrchestrator), ajan implementasyonları ve harici sistemlere bağlanan MCP adaptör katmanından oluşur. Dağıtım hedefi, Oracle Cloud Infrastructure (OCI) Ücretsiz Sürüm kaynakları üzerinde düşük bağımlılıkla çalışabilen bir prototiptir.

Kapsama dahil olan ana bileşenler aşağıda özetlenmiştir:

- **Platform çekirdeği:** job oluşturma ve izleme, durum yönetimi, log ve iz kaydı, çıktı (artefakt) saklama.
- **Ajanlar:** AKIS Scribe, AKIS Trace ve AKIS Proto için asgari işlevsellik ve yapılandırma parametreleri.
- **Entegrasyonlar:** GitHub/Jira/Confluence gibi sistemlere Model Context Protocol (MCP) adaptörleri üzerinden erişim.
- **Kimlik doğrulama ve onboarding:** e-posta tabanlı çok adımlı kayıt ve giriş akışları, stateless JWT oturumu.
- **Akademik çıktılar:** mimari açıklamalar ve proje rapor seti.

Kapsam dışı tutulan başlıca unsurlar aşağıdadır:

- **Kodsuz/düşük kodlu ajan geliştirme:** sihirbazları veya kullanıcı tarafından programlanabilir ajan mantığı.
- **Kurumsal ölçekte gelişmiş ekip yönetimi:** çok katmanlı RBAC ve on-premise dağıtım seçenekleri.
- **Üretim seviyesinde dağıtım:** Kubernetes tabanlı dağıtım ve kapsamlı kurumsal CI/CD otomasyonu.
- **Ödeme/faturalandırma:** modülleri ve çok katmanlı abonelik altyapısı.

1.3. Terimler ve Kısaltmalar

Terim	Açıklama
AKIS	LLM tabanlı ajanlar ile etkileşimli yazılım geliştirme platformu.
LLM	Large Language Model (Büyük Dil Modeli).
MCP	Model Context Protocol; harici sistemlere standartlaştırılmış bağlanma katmanı.
OCI	Oracle Cloud Infrastructure.
JWT	JSON Web Token; HTTP-only cookie üzerinden stateless oturum.
SDLC	Software Development Life Cycle (Yazılım Geliştirme Yaşam Döngüsü).

2. İLGİLİ ÇALIŞMALAR / LİTERATÜR ARAŞTIRMASI

2.1. Araştırma Metodolojisi

Bu çalışmada, LLM tabanlı otonom ajanların yazılım mühendisliği süreçlerinde kullanımı, otomatik dokümantasyon üretimi, test üretimi ve doğrulama mekanizmaları üzerine güncel araştırmalar incelenmiştir. Tarama; IEEE Xplore, ACM Digital Library, SpringerLink ve Google Scholar gibi veri tabanlarında, 2020-2025 aralığındaki hakemli çalışmalarla sınırlandırılmıştır. Bulgular; otonom ajan mimarileri, dokümantasyon üretimi, test üretimi ve akıl yürütme stratejileri başlıkları altında sınıflandırılarak sentezlenmiştir.

2.2. Bulguların Özeti ve AKIS ile İlişkisi

Literatürde, LLM tabanlı ajanların planlama ve araç kullanımıyla gerçek kod tabanları üzerinde görev yürütme kabiliyetleri gösterilmiştir [1]-[5]. Otomatik dokümantasyon üretimi alanında, kod açıklaması ve dokümantasyon güncelleme kalitesi üzerine karşılaştırmalı çalışmalar mevcuttur [6]-[8]. Test üretimi ve hata düzeltme tarafında ise LLM destekli test üretimi, benchmark setleri ve güvenilirlik değerlendirmeleri öne çıkmaktadır [9]-[12]. ReAct, Chain-of-Thought ve Tree-of-Thoughts gibi yaklaşımlar, çok adımlı görevlerde daha kararlı planlama ve eylem üretimi için temel oluşturmaktadır [13]-[15].

AKIS Platformu, literatürde çoğunlukla tekil görev odaklı kalan yaklaşımlardan farklı olarak; dokümantasyon, test ve prototipleme işlevlerini tek bir orkestrasyon çekirdeği ve standart entegrasyon katmanı altında birleştirmeyi hedefler. Bu bütünleşik yapı, literatürde önerilen planla-yürüt-eleştir/doğrula türündeki güvenilirlik adımlarını platform düzeyinde sistematik hale getirecek şekilde tasarlanmıştır.

3. PROJE GEREKSİNİMLERİ

3.1. Fonksiyonel Gereksinimler

Kod	Açıklama
FR1	Kullanıcı kayıt ve giriş akışları (çok adımlı) sağlanacaktır.
FR2	Kullanıcı, ajan konfigürasyonunu UI üzerinden oluşturup kaydedebilecektir.
FR3	Kullanıcı, bir ajan işi (job) başlatabilecek ve durumunu izleyebilecektir.
FR4	Scribe ajanı; kaynak depodaki değişikliği analiz edip dokümantasyon güncelleme çıktısı üretecektir.
FR5	Trace ajanı; gereksinim ve kabul kriterlerinden test senaryosu ve otomasyon çıktısı üretecektir.
FR6	Proto ajanı; gereksinim girdisinden analiz-tasarım-prototip akışını yürüterek artefaktlar üretecektir.
FR7	Harici sistemlere erişim, MCP adaptörleri üzerinden standartlaştırılmış şekilde sağlanacaktır.

3.2. Fonksiyonel Olmayan Gereksinimler

Kod	Açıklama
NFR1	Sistem, OCI Ücretsiz Sürüm kaynaklarında (düşük CPU/RAM) çalışacak şekilde tasarlanacaktır.
NFR2	Kimlik doğrulama, HTTP-only cookie içinde stateless JWT ile sağlanacaktır.
NFR3	Girdi doğrulama ve sözleşme uyumluluğu çalışma zamanında doğrulanacaktır.
NFR4	İş yürütme adımları ve durum değişimleri denetlenebilir kayıtlarla saklanacaktır.
NFR5	Harici entegrasyonlar doğrudan vendor SDK'ları ile değil, MCP adaptör katmanı ile yapılacaktır.
NFR6	Sistem; hata yönetimi, zaman aşımı ve yeniden deneme stratejileri ile kararlı çalışacaktır.

3.3. Tasarım Kısıtları ve Varsayımlar

Bu dokümanda tasarım kararlarını belirleyen temel kısıtlar aşağıdadır:

Backend: Fastify + TypeScript; ORM: Drizzle; veritabanı: PostgreSQL.

Frontend: React + Vite SPA; stil: Tailwind CSS.

Dış sistem entegrasyonları: MCP adaptörleri aracılığıyla yapılacaktır.

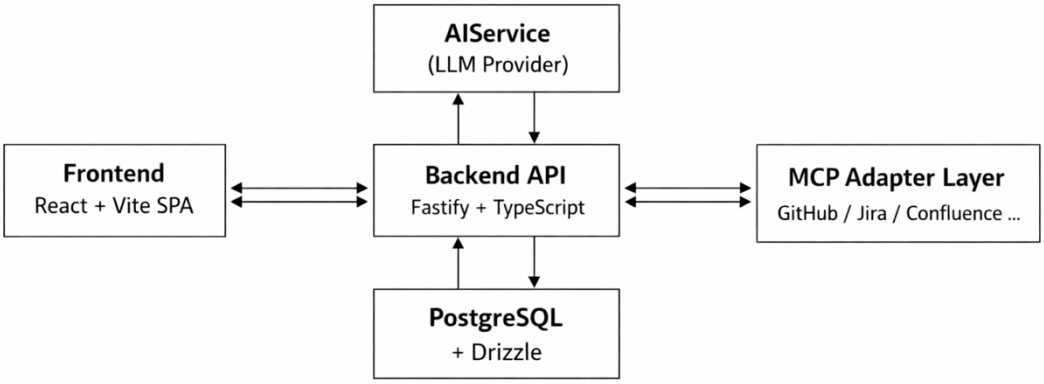
Oturum yönetimi: 7 günlük HTTP-only cookie içinde JWT; parolalar bcrypt ile hashlenir.

Kaynak verimliliği: uygulama ve veritabanı aynı VM üzerinde çalışacak şekilde basit dağıtım hedeflenir.

4. SİSTEM TASARIMI

4.1. Genel Mimari Bakış

AKIS Platformu, tek repo içinde modüler monolit bir backend ve bağımsız bir SPA frontend yaklaşımıyla tasarlanmıştır. Backend; API katmanı, çekirdek orkestrasyon (AgentOrchestrator), ajan implementasyonları ve servis/adaptör katmanları olarak ayrıştırılmıştır. Frontend; kullanıcı onboarding akışları, ajan konfigürasyonu ve job çalıştırma/izleme ekranlarından oluşur.



Şekil 4.1: AKIS Platformu genel mimari bileşenleri (backend–frontend–veritabanı–MCP entegrasyonları).

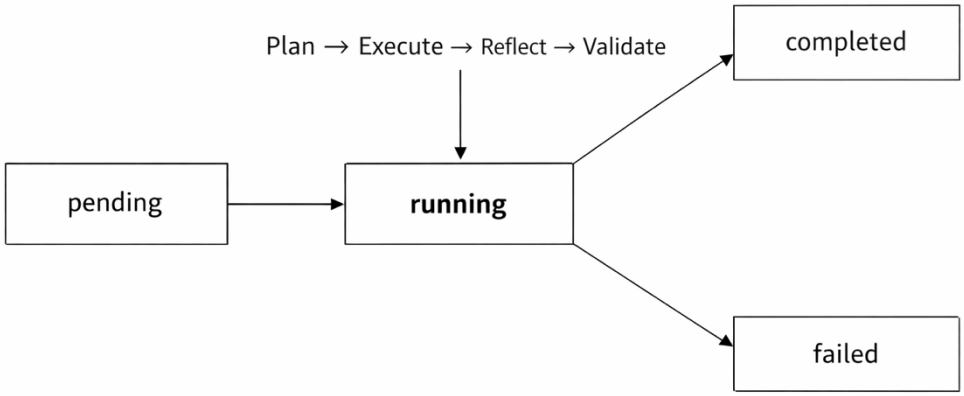
4.2. Backend Tasarımı

4.2.1. AgentOrchestrator ve Ajan Yaşam Döngüsü

Merkezi düzenleyici (AgentOrchestrator), ajanların birbirini çağırmadan tek bir kontrol noktasından çalıştırılmasını sağlar. Orchestrator; job başlatma, durum yönetimi (pending -> running -> completed/failed), bağımlılıkların enjeksiyonu (DB, MCP adaptörleri, AIService), hata yakalama ve gerektiğinde tekrar deneme stratejilerinden sorumludur.

Ajan iş akışları; planlama (planner), yürütme (executor), eleştiri/iyileştirme (reflection) ve doğrulama adımlarını içerir. Bu yaklaşım, çıktı kalitesini artırırken; insan onayı gerektiren noktalarda güvenli operasyonu destekler.

Planlama aşaması; görevin alt adımlara ayrılması ve araç kullanım stratejisinin belirlenmesini kapsar. Yürütme aşaması; MCP adaptörleri, HTTP istemcisi ve AIService gibi araçların enjekte edilmesiyle planın adım adım çalıştırılmasıdır. Eleştiri/iyileştirme aşaması; üretilen çıktının tutarlılık ve gereksinim uyumu açısından yeniden değerlendirilmesi ve gerekirse düzeltme istemesidir. Doğrulama ise, çıktıların format ve içerik sözleşmelerine uygunluğunun kontrol edilmesi ve job sonuçlarının artefakt olarak saklanmasını içerir.



Şekil 4.2: Job yaşam döngüsü için durum makinesi (FSM) özeti.

4.2.2. Kimlik Doğrulama ve Onboarding Akışları

Kimlik doğrulama, e-posta ve parola temelli çok adımlı kayıt ve giriş akışları üzerinden sağlanır. Oturum, HTTP-only cookie içinde stateless JWT ile yönetilir. E-posta doğrulama, 6 haneli kod ve 15 dakika süreli token modeli ile yürütülür; oran sınırlama ve kötüye kullanım kontrolleri uygulanır.

4.2.3. API Katmanı ve Uç Nokta Özeti

API katmanı; kimlik doğrulama, ajan job yönetimi ve sistem sağlık uçlarını sağlayan REST uçlarından oluşur. Aşağıdaki tabloda, temel uç noktalar ve amaçları özetlenmiştir.

Method	Path	Amaç
GET	/health, /ready, /version	Sağlık kontrolleri ve sürüm bilgisi.
POST	/auth/signup/start; /auth/signup/password; /auth/verify-email	Kayıt ve e-posta doğrulama akışları.
POST	/auth/login/start; /auth/login/complete; /auth/logout	Giriş ve çıkış akışları.
GET	/auth/me	Mevcut kullanıcı oturum bilgisi.
POST/GET	/api/agents/jobs	Job oluşturma ve listeleme.
GET	/api/agents/jobs/:id	Job detay görüntüleme (durum, plan, iz, artefakt).

4.2.4. Güvenlik Katmanı ve Orta Katmanlar

Uygulama katmanında güvenlik; oturum çerezi (HTTP-only), e-posta doğrulama kodları ve oran sınırlama kontrolleri etrafında kurgulanmıştır. Kimlik doğrulama akışlarında 6 haneli kodların süreli olması, deneme sayılarının kısıtlanması ve hatalı denemelerde gecikme uygulanması hedeflenmiştir. HTTP katmanında; CORS politikası, güvenli başlıklar ve temel rate-limit önlemleri ile kötüye kullanım riskleri azaltılmaktadır. Ayrıca, log/iz kayıtlarında gizli anahtarların maskelemesi ve hata mesajlarının kontrollü olarak dönülmesi, güvenli işletim açısından önemlidir.

4.2.5. Modül Yapısı ve Dosya Organizasyonu

Backend, tek bir repository içinde modüler monolit prensibiyle katmanlara ayrılmıştır. API yönlendirmeleri ve route tanımları src/api/ altında toplanırken; ajan yaşam döngüsünü yöneten orkestrasyon çekirdeği src/core/orchestrator/ altında konumlandırılmıştır. Ajan sözleşmeleri, durum makinesi ve ortak playbook yapıları ilgili çekirdek dizinlerinde yönetilir. Dış sistem entegrasyonları ve AI sağlayıcı erişimi; servis/adaptör katmanı olarak izole edilmiştir. Bu organizasyon, kod tabanının büyümesini yönetilebilir kılmayı ve her katmanın sorumluluğunu ayrıştırmayı hedefler.

4.3. Frontend Tasarımı

4.3.1. Bilgi Mimarisi ve Ekranlar

Frontend, React + Vite tabanlı SPA olarak geliştirilmiş ve aşağıdaki ana ekranları içermektedir:

Kimlik doğrulama sihirbazı: kayıt (çok adım), giriş (iki adım), gizlilik onayı.

Dashboard: ajan seçimi, konfigürasyon paneli, job oluşturma ve izleme.

Job detay ekranı: durum, iz kayıtları, çıktı artefaktları ve revizyon akışı.

4.4. Veri Modeli Tasarımı

Platformun temel veri varlıkları aşağıda özetlenmiştir:

Aşağıdaki tabloda platformun temel veritabanı tabloları ve rolleri özetlenmiştir.

Tablo	Amaç
users	Kullanıcı hesabı, doğrulama ve tercih alanları.
email_verification_tokens	6 haneli doğrulama kodları ve süre/kullanım durumu.
jobs	Ajan işleri için ana kayıt; durum, tip ve meta veriler.
job_plans	Planner çıktısı ve adım planları.
job_traces	Ajan çalışma izleri ve zaman damgaları.
job_artifacts	Üretilen dosyalar/çıktılar ve bağlantıları.
agent_configs	Ajan başına kullanıcı konfigürasyonu.

4.4.1. Varlıklar ve İlişkisel Model Özeti

Veri modeli; kullanıcı kimliği ve oturum yönetimi, ajan konfigürasyonlarının saklanması ve ajan job yürütmelerinin denetlenebilir şekilde kaydedilmesi üzerine kuruludur. Job kayıtları; plan, iz kaydı (trace), artefakt ve denetim (audit) varlıklarıyla ilişkili olacak şekilde tasarlanmıştır. Bu yapı, hem yeniden üretilebilirlik hem de hata analizi için gerekli bağlamın kalıcılaştırılmasını sağlar.

4.4.2. Job, İz Kaydı ve Artefakt Yaşam Döngüsü

Platformun çekirdek işleyişi, ‘job’ kavramı etrafında şekillenir. Her job; ajan tipi, kullanıcı konfigürasyonu ve giriş parametreleri ile başlar; yürütme sırasında üretilen plan adımları, iz kayıtları ve artefaktlar veritabanına eklenir. Bu yaklaşım, ajanın hangi araçları hangi sırayla kullandığını ve hangi çıktıyı ürettiğini sonradan incelemeyi

mümkün kılar. Job zinciri (revizyon) desteği ile aynı görevin farklı revizyonları arasında ilişki kurulabilir.

4.4.3. Tablo Alanları (Kolon Bazında)

Bu alt bölümde, SDTA teknik snapshot çıktısına dayalı olarak temel tabloların alanları özetlenmiştir. Tablo alanları, iş akışlarının izlenebilirliğini ve denetlenebilirliğini desteklemek üzere seçilmiştir.

users — Kullanıcılar

Column	Type	Purpose
id	UUID	Primary key
name	TEXT	Full name
email	TEXT	Email (unique)
passwordHash	TEXT	bcrypt hash (12 rounds)
status	ENUM	pending_verification/active/disabled/deleted
emailVerified	BOOLEAN	Email verification flag
dataSharingConsent	BOOLEAN	Consent flag (nullable)
hasSeenBetaWelcome	BOOLEAN	Beta welcome screen flag
createdAt	TIMESTAMP	Creation time

updatedAt	TIMESTAMP	Last update time

email_verification_tokens — E-posta doğrulama kodları

Column	Type	Purpose
id	UUID	Primary key
userId	UUID	FK to users (cascade delete)
email	TEXT	Email address
code	VARCHAR(6)	6-digit verification code
expiresAt	TIMESTAMP	Code expiry time (15 minutes)
usedAt	TIMESTAMP	When code was used (nullable)
createdAt	TIMESTAMP	Creation time

oauth_accounts — OAuth hesapları

Column	Type	Purpose
id	UUID	Primary key
userId	UUID	FK to users (cascade delete)

provider	ENUM	github/google
providerAccountId	TEXT	OAuth provider user ID
accessToken	TEXT	OAuth access token (stored securely)
refreshToken	TEXT	OAuth refresh token (nullable)
tokenExpiresAt	TIMESTAMP	Token expiry time (nullable)
createdAt	TIMESTAMP	Creation time
updatedAt	TIMESTAMP	Last update time

agent_configs — Ajan konfigürasyonları

Column	Type	Purpose
id	UUID	Primary key
userId	UUID	FK to users (cascade delete)
agentType	VARCHAR(50)	scribe/trace/proto
enabled	BOOLEAN	Agent enabled flag
repositoryOwner	VARCHAR(255)	GitHub owner (for GitHub-dependent agents)

repositoryName	VARCHAR(255)	GitHub repository name
baseBranch	VARCHAR(255)	Base branch (default: main)
branchPattern	VARCHAR(255)	Branch naming pattern (default: docs/{agent}-{timestamp})
targetPlatform	VARCHAR(50)	confluence/notion/github_wiki/github_repo
targetConfig	JSONB	Target-specific configuration (flexible)
triggerMode	VARCHAR(50)	on_pr_merge/scheduled/manual (default: manual)
scheduleCron	VARCHAR(100)	Cron expression for scheduled triggers
prTitleTemplate	VARCHAR(500)	PR title template (default: docs(scribe): update {path})
prBodyTemplate	TEXT	PR body template
autoMerge	BOOLEAN	Auto-merge PR flag (default: false)
includeGlobs	TEXT[]	Include glob patterns
excludeGlobs	TEXT[]	Exclude glob patterns

jobTimeoutSeconds	INTEGER	Job timeout (default: 60)
maxRetries	INTEGER	Max retry count (default: 2)
llmModelOverride	VARCHAR(255)	LLM model override (optional)
createdAt	TIMESTAMP	Creation time
updatedAt	TIMESTAMP	Last update time

jobs — Job kayıtları

Column	Type	Purpose
id	UUID	Primary key
type	VARCHAR(50)	Agent type (scribe/trace/proto)
state	ENUM	Job state (pending/running/completed/failed/awaiting_approval)
payload	JSONB	Input parameters
result	JSONB	Execution output
error	VARCHAR(1000)	Error message (truncated)

errorCode	VARCHAR(50)	Structured error code (e.g., MCP_UNREACHABLE)
errorMessage	VARCHAR(500)	User-friendly error message
rawErrorPayload	TEXT	Full error JSON (for debugging)
mcpGatewayUrl	VARCHAR(255)	MCP Gateway used for job
requiresStrictValidation	BOOLEAN	Enable strong model validation
requiresApproval	BOOLEAN	Requires human approval before execution
approvedBy	UUID	User who approved job (FK to users)
approvedAt	TIMESTAMP	Approval timestamp
rejectedBy	UUID	User who rejected job (FK to users)
rejectedAt	TIMESTAMP	Rejection timestamp
approvalComment	TEXT	Approval/rejection comment
parentJobId	UUID	Parent job for revision chain
revisionNote	TEXT	Revision instruction
createdAt	TIMESTAMP	Creation time

updatedAt	TIMESTAMP	Last update time

job_plans — Job plan adımları

Column	Type	Purpose
id	UUID	Primary key
jobId	UUID	FK to jobs (cascade delete)
steps	JSONB	Plan steps array (legacy)
rationale	TEXT	Planning rationale (legacy)
planMarkdown	TEXT	Contract-first plan document (Markdown)
planJson	JSONB	Structured plan data (JSON)
createdAt	TIMESTAMP	Creation time
updatedAt	TIMESTAMP	Last update time

job_traces — İz kayıtları (trace)

Column	Type	Purpose
id	UUID	Primary key

jobId	UUID	FK to jobs (cascade delete)
eventType	ENUM	step_start/step_complete/doc_read/file_created/mcp_call/ai_call/error/info/tool_call/decision/reasoning
stepId	VARCHAR(100)	Plan step ID (if applicable)
title	VARCHAR(500)	Event summary
detail	JSONB	Event details (raw, server-side only)
durationMs	INTEGER	Duration in milliseconds
status	VARCHAR(20)	success/failed/warning/info
correlationId	VARCHAR(100)	MCP correlation ID
gatewayUrl	VARCHAR(500)	Gateway URL for MCP events
errorCode	VARCHAR(50)	Error code if failed
timestamp	TIMESTAMP	Event timestamp

toolName	VARCHAR(100)	Tool name for tool_call events
inputSummary	TEXT	Input arguments summary (redacted)
outputSummary	TEXT	Output result summary (redacted)
reasoningSummary	VARCHAR(1000)	User-facing reasoning (2-4 sentences)
askedWhat	TEXT	"Asked" - what agent asked tool to do
didWhat	TEXT	"Did" - what action was taken
whyReason	TEXT	"Why" - why this action was taken

job_artifacts — Üretilen artefaktlar

Column	Type	Purpose
id	UUID	Primary key
jobId	UUID	FK to jobs (cascade delete)
artifactType	VARCHAR(50)	doc_read/file_created/file_modified
path	VARCHAR(1000)	File path or document identifier

operation	VARCHAR(20)	read/create/modify/delete
sizeBytes	INTEGER	File size (if applicable)
contentHash	VARCHAR(64)	SHA-256 hash (for deduplication)
preview	TEXT	Preview/excerpt (truncated, max 1KB)
metadata	JSONB	Additional metadata
diffPreview	TEXT	Unified diff preview (for modified files)
linesAdded	INTEGER	Number of lines added
linesRemoved	INTEGER	Number of lines removed
createdAt	TIMESTAMP	Creation time

4.5. Entegrasyon Tasarımı

Harici sistem entegrasyonları, MCP adaptör katmanı üzerinden standartlaştırılmıştır. Bu yaklaşım; farklı sağlayıcılar ile etkileşimde protokol çeşitliliğini tek bir sözleşme altında toplar, ajanın araç kullanımını daha güvenli ve izlenebilir hale getirir ve vendor değişimlerine karşı dayanıklılığı artırır.

4.5.1. MCP Gateway ve Adaptör Yaklaşımı

MCP Gateway; harici sistemlerle (ör. GitHub, doküman depoları, görev yönetim araçları) konuşan adaptörleri tek bir giriş noktası altında toplar. Backend tarafında AgentOrchestrator, ajan adımlarında ihtiyaç duyduğu 'tool' çağrılarını bu gateway'e iletir; gateway ise ilgili adaptöre yönlendirerek standart bir yanıt formatı döndürür. Böylece entegrasyon kodu çekirdek orkestrasyon katmanından ayrıştırılır ve yeni bir

entegrasyon eklemek için ajan mantığını değiştirmek yerine yeni bir adaptör eklemek yeterli olur.

Bu yapı; zaman aşımı, hata sınıflandırması ve log/iz üretimi gibi çapraz-kesit (cross-cutting) ihtiyaçları gateway seviyesinde ele almayı sağlar. Örneğin; bir GitHub API hatası, gateway üzerinde normalize edilerek ajanın 'Reflect/Validate' adımlarında tutarlı şekilde ele alınabilir.

4.5.2. Docker Bileşenleri ve Çalıştırma Modeli

Prototip, yerel geliştirme ve demo dağıtımı için Docker tabanlı bileşen ayrımı ile çalıştırılabilir şekilde tasarlanmıştır. Tipik bir kurulumda; PostgreSQL (kalıcılık), backend API (Fastify) ve MCP Gateway ayrı konteynerler olarak çalışır. Frontend çıktısı statik olarak servis edilebilir veya geliştirme aşamasında Vite dev sunucusu ile ayrık çalıştırılabilir.

Bu yaklaşım; bağımlılıkların sürüm uyumluluğunu korur, tek komutla ayağa kalkabilen bir demo ortamı sağlar ve OCI üzerinde taşınabilir bir dağıtım modeli sunar. Sağlık uçları (healthcheck) ve log yönlendirmesi, temel işletim ihtiyaçları için başlangıç seviyesi gözlemlenebilirlik sağlar.

4.5.3. Entegrasyon Güvenliği ve Erişim Kontrolleri

Entegrasyon anahtarları ve erişim belirteçleri (token) çevresel değişkenler üzerinden yönetilir; log/iz kayıtlarında gizli değerlerin maskelemesi hedeflenir. Gateway ve backend arasında ağ erişimi; sadece gerekli portlar üzerinden sınırlandırılabilir. Ayrıca, oran sınırlama (rate limiting) ve istek doğrulama kontrolleri ile kötüye kullanım riskleri azaltılır. Job iz kayıtları, kullanıcı bazlı yetkilendirme ile erişime açılarak çok kullanıcılı senaryolarda izolasyon sağlanır.

4.6. Test Planı ve Kalite Stratejisi

Kalite güvencesi yaklaşımı, erken test ve sözleşme doğrulaması üzerine kuruludur:

Birim testleri: orkestrasyon akışları, validasyonlar ve servis katmanı için izole testler.

Sözleşme testleri: ajan girdi/çıktı şemalarının doğrulanması.

Entegrasyon testleri: MCP adaptörlerinin protokol uyumluluğu ve hata senaryoları.

Uçtan uca testler: kritik kullanıcı akışları (kayıt, giriş, job oluşturma, izleme).

Test planı; otomasyonun yanı sıra manuel doğrulama adımlarını da içerir: kritik ekran akışlarının kabul kriterlerine göre kontrolü ve örnek repo üzerinde Scribe/Trace çıktılarının gözden geçirilmesi.

4.7. Dağıtım ve İşletim

Dağıtım hedefi OCI Ücretsiz Sürüm olduğu için, tek süreçli modüler monolit yaklaşımı tercih edilmiştir. Uygulama ve PostgreSQL aynı VM üzerinde çalıştırılarak operasyonel basitlik sağlanır. Sağlık uç noktaları (/health, /ready) ve yapılandırılmış loglama ile temel işletim görünürlüğü elde edilir.

5. TAMAMLANAN GÖREVLER

5.1. Yapılan İşler ve Ara Çıktılar

Proje kapsamı ve gereksinimler dokümanında tanımlanan proje yönetimi yaklaşımına uygun olarak, bu fazda aşağıdaki işler tamamlanmıştır:

Modüler monolit backend temel altyapısı (Fastify + TypeScript) ve temel API iskeleti oluşturuldu.

PostgreSQL + Drizzle veri erişim katmanı ve çekirdek tablo seti kuruldu.

Kimlik doğrulama akışları (çok adımlı kayıt/giriş, e-posta doğrulama, oturum) geliştirildi.

Ajan orkestrasyon çekirdeği (AgentOrchestrator), durum yönetimi ve temel job yaşam döngüsü kurgulandı.

Scribe ajanı için konfigürasyon ve dashboard üzerinden job çalıştırma/izleme akışı prototip seviyesinde oluşturuldu; demo senaryoları için temel çıktı üretimi ve iz kaydı hazırlandı.

MCP adaptör katmanı için entegrasyon çerçevesi ve ilk bağlantı senaryoları hazırlandı.

Kalite stratejisi kapsamında; birim ve entegrasyon seviyesinde testler ile temel kullanıcı akışlarını doğrulayan kontroller oluşturulmuş, CI hattında otomatik

çalıştırma hedeflenmiştir. Mevcut aşamada 130+ test senaryosu ile regresyon riskinin azaltılması amaçlanmaktadır.

5.2. Güncel Durum Tablosu

Aşağıdaki tabloda, proje fazları 0.1'den 1.0'a kadar tek bir formatta özetlenmiştir. Durum ifadeleri; tamamlanmış işler, demo hazırlığı ve devam eden geliştirmeleri gösterecek şekilde verilmiştir.

Faz	Odak	Durum
Faz 0.1	Repo iskeleti ve temel proje kurulumu	Tamamlandı
Faz 0.2	Backend API altyapısı (Fastify) ve sağlık uçları	Tamamlandı
Faz 0.3	Veritabanı şeması ve Drizzle erişim katmanı	Tamamlandı
Faz 0.4	Kimlik doğrulama ve onboarding akışları	Tamamlandı
Faz 0.5	AgentOrchestrator çekirdeği ve job yaşam döngüsü (FSM)	Tamamlandı
Faz 0.6	Scribe konfigürasyon dashboard ve job izleme (demo akışı)	Demo Hazır
Faz 1.0	Scribe job uçtan uca (artefakt üretimi ve doğrulama)	Devam Ediyor

KAYNAKLAR

[1] Suri, S., Das, S. N., Singi, K., Dey, K., Sharma, V. S., & Kaulgud, V. (2023). Software engineering using autonomous agents: Are we there yet? ASE 2023.

[2] Bouzenia, I., Devanbu, P., & Pradel, M. (2025). RepairAgent: An autonomous, LLM-based agent for program repair. ICSE 2025.

[3] He, J., Treude, C., & Lo, D. (2025). LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. ACM TOSEM, 34(5).

[4] Yang, J., et al. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024.

[5] Li, G., et al. (2023). CAMEL: Communicative agents for “mind” exploration of large language model society. NeurIPS 2023.

[6] Khan, J. Y., & Uddin, G. (2023). Automatic code documentation generation using GPT-3. ASE 2022.

[7] Dvivedi, S. S., et al. (2024). A comparative analysis of large language models for code documentation generation. AIware 2024.

[8] Yang, D., et al. (2025). DocAgent: A multi-agent system for automated code documentation generation. ACL 2025 (System Demonstrations).

[9] Wang, J., et al. (2024). Software testing with large language models: Survey, landscape, and vision. IEEE TSE, 50(4), 911-936.

[10] Bhatia, S., et al. (2024). Unit test generation using generative AI: A comparative performance analysis of autogeneration tools. LLM4Code 2024.

[11] Tony, C., et al. (2023). LLMSecEval: A dataset of natural language prompts for security evaluations. MSR 2023.

[12] Mündler, N., et al. (2024). SWT-Bench: Testing and validating real-world bug-fixes with code agents. NeurIPS 2024.

[13] Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR 2023.

[14] Wang, X., et al. (2023). Self-consistency improves chain-of-thought reasoning in language models. ICLR 2023.

[15] Yao, S., et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. NeurIPS 2023.